

1.

```
SELECT emp_name, department, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```
2.

```
SELECT emp_id, emp_name, department, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```
3.

```
SELECT emp_id, emp_name, department, salary
FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```
4.

```
SELECT emp_name, department, hire_year
FROM employees
WHERE hire_year = (SELECT hire_year FROM employees WHERE emp_name = 'Rachel King')
AND emp_name <> 'Rachel King';
```
5.

```
SELECT product_name, category, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```
6.

```
SELECT product_id, product_name, category, price
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```
7.

```
SELECT product_id, product_name, category, price
FROM products
WHERE price = (SELECT MIN(price) FROM products);
```
8.

```
SELECT order_id, customer_id, amount
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders)
ORDER BY amount DESC;
```
9.

```
SELECT order_id, customer_id, amount, order_date
FROM orders
WHERE amount = (SELECT MAX(amount) FROM orders);
```
10.

```
SELECT customer_id, customer_name, city
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders)
ORDER BY customer_id;
```
11.

```
SELECT customer_id, customer_name, city
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders)
ORDER BY customer_id;
```

12.

```
SELECT product_id, product_name, category, price
FROM products
WHERE product_id IN (SELECT product_id FROM orders);
```

13.

```
SELECT product_id, product_name, category, price
FROM products
WHERE product_id NOT IN (SELECT product_id FROM orders);
```

14.

```
SELECT emp_name, salary
FROM employees
WHERE department IN (
    SELECT dept_name FROM departments WHERE dept_name = 'Finance'
)
ORDER BY salary DESC;
```

15.

```
SELECT emp_name, department, salary
FROM employees
WHERE salary < (
    SELECT MIN(salary) FROM employees WHERE department = 'Finance'
)
ORDER BY salary DESC;
```

16.

```
SELECT dept_name, budget, location
FROM departments
WHERE budget = (SELECT MAX(budget) FROM departments);
```

17.

```
SELECT customer_id, customer_name, city
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
    GROUP BY customer_id
    HAVING COUNT(*) = 1
);
```

18.

```
SELECT customer_id, customer_name, city
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
    GROUP BY customer_id
    HAVING COUNT(*) >= 2
)
ORDER BY customer_id;
```

19.

```
SELECT product_name, category, price
FROM products
WHERE price > (
    SELECT MAX(price) FROM products WHERE category = 'Accessories'
)
ORDER BY price DESC;
```

20.

```
SELECT order_id, customer_id, amount, order_date
FROM orders
WHERE customer_id IN (
    SELECT customer_id FROM customers WHERE join_year = 2021
)
ORDER BY order_date;
```

21.

```
SELECT e.emp_id, e.emp_name, e.department
FROM employees e
WHERE EXISTS (
    SELECT 1 FROM employees x WHERE x.manager_id = e.emp_id
)
ORDER BY e.emp_name;
```

22.

```
SELECT e.emp_id, e.emp_name, e.department
FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM employees x WHERE x.manager_id = e.emp_id
)
ORDER BY e.emp_id;
```

23.

```
SELECT c.customer_id, c.customer_name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

24.

```
SELECT c.customer_id, c.customer_name, c.city
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

25.

```
SELECT p.product_id, p.product_name, p.category
FROM products p
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.product_id = p.product_id
)
ORDER BY p.product_id;
```

26.

```
SELECT emp_name, department, salary
FROM employees
WHERE department = (
    SELECT department FROM employees WHERE emp_id = 105
)
AND emp_id <> 105
ORDER BY salary DESC;
```

27.

```

SELECT order_id, customer_id, amount, order_date
FROM orders
WHERE order_date = (SELECT MAX(order_date) FROM orders);

```

28.

```

SELECT order_id, customer_id, amount, order_date
FROM orders
WHERE order_date = (SELECT MIN(order_date) FROM orders);

```

29.

```

SELECT emp_name, department, salary
FROM employees
WHERE salary > (
    SELECT MAX(salary) FROM employees WHERE department = 'HR'
)
ORDER BY salary DESC;

```

30.

```

SELECT order_id, customer_id, amount, status
FROM orders
WHERE status = 'Delivered'
AND amount > (
    SELECT AVG(amount) FROM orders WHERE status = 'Delivered'
)
ORDER BY amount DESC;

```

31.

```

SELECT e.emp_name, e.department, e.salary,
       ROUND((SELECT AVG(x.salary) FROM employees x
              WHERE x.department = e.department), 2) AS dept_avg
FROM employees e
WHERE e.salary > (
    SELECT AVG(x.salary) FROM employees x
    WHERE x.department = e.department
)
ORDER BY e.department, e.salary DESC;

```

32.

```

SELECT e.emp_name, e.department, e.salary,
       (SELECT COUNT(*) FROM employees x
        WHERE x.department = e.department
        AND x.salary > e.salary) AS higher_earners
FROM employees e
ORDER BY e.department, e.salary DESC;

```

33.

```

SELECT c.customer_id, c.customer_name,
       (SELECT SUM(o.amount) FROM orders o
        WHERE o.customer_id = c.customer_id) AS total_spend
FROM customers c
WHERE (SELECT SUM(o.amount) FROM orders o
       WHERE o.customer_id = c.customer_id) > (
    SELECT AVG(total_spend)
    FROM (
        SELECT customer_id, SUM(amount) AS total_spend
        FROM orders GROUP BY customer_id
    ) x
)

```

```
)  
ORDER BY total_spend DESC;
```

34.

```
SELECT emp_name, department, salary  
FROM employees  
WHERE department IN (  
    SELECT department  
    FROM employees  
    GROUP BY department  
    HAVING AVG(salary) > 65000  
)  
ORDER BY department, salary DESC;
```

35.

```
SELECT o.order_id, o.customer_id, o.amount,  
       CASE  
           WHEN o.amount >= (  
               SELECT AVG(x.amount) FROM orders x  
               WHERE x.customer_id = o.customer_id  
           ) THEN 'Above Avg'  
           ELSE 'Below Avg'  
       END AS vs_cust_avg  
FROM orders o  
ORDER BY o.customer_id, o.order_id;
```

36.

```
SELECT p.product_name, p.category, p.price,  
       ROUND((SELECT AVG(x.price) FROM products x  
              WHERE x.category = p.category), 2) AS cat_avg  
FROM products p  
WHERE p.price > (  
    SELECT AVG(x.price) FROM products x  
    WHERE x.category = p.category  
)  
ORDER BY p.category, p.price DESC;
```

37.

```
SELECT emp_name, department, salary  
FROM employees  
WHERE salary > ANY (  
    SELECT salary FROM employees  
    WHERE department = 'Marketing'  
)  
ORDER BY salary DESC;
```

38.

```
SELECT emp_name, department, salary  
FROM employees  
WHERE salary < ALL (  
    SELECT salary FROM employees  
    WHERE department = 'Finance'  
)  
ORDER BY salary DESC;
```

39.

```
SELECT c.customer_id, c.customer_name
```

```

FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
)
AND NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.status <> 'Delivered'
)
ORDER BY c.customer_id;

```

40.

```

SELECT product_id, product_name, category
FROM products
WHERE product_id IN (
    SELECT product_id FROM orders
    WHERE customer_id IN (
        SELECT customer_id FROM customers
        WHERE city = 'Mumbai'
    )
)
ORDER BY product_id;

```

41.

```

SELECT emp_name, department, salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees)
);

```

42.

```

SELECT DISTINCT e.department
FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM employees x
    WHERE x.department = e.department
    AND x.salary <= 50000
)
ORDER BY e.department;

```

43.

```

SELECT c.customer_id, c.customer_name
FROM customers c
WHERE (
    SELECT COUNT(DISTINCT o.product_id)
    FROM orders o
    WHERE o.customer_id = c.customer_id
) >= 2
ORDER BY c.customer_id;

```

44.

```

SELECT e.emp_name, e.department, e.salary, e.hire_year,
    ROUND((SELECT AVG(x.salary) FROM employees x

```

```

        WHERE x.hire_year = e.hire_year), 2) AS year_avg
FROM employees e
WHERE e.salary > (
    SELECT AVG(x.salary) FROM employees x
    WHERE x.hire_year = e.hire_year
)
ORDER BY e.hire_year, e.salary DESC;

```

45.

```

SELECT p.product_name, p.category, p.stock_qty,
       ROUND((SELECT AVG(x.stock_qty) FROM products x
              WHERE x.category = p.category), 2) AS cat_avg_stock
FROM products p
WHERE p.stock_qty < (
    SELECT AVG(x.stock_qty) FROM products x
    WHERE x.category = p.category
)
ORDER BY p.category, p.stock_qty ASC;

```

46.

```

SELECT emp_name, department, salary
FROM employees
WHERE salary IN (
    SELECT DISTINCT salary
    FROM employees
    ORDER BY salary DESC
    LIMIT 3
)
ORDER BY salary DESC;

```

47.

```

SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
)
AND NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.amount <= 1000
)
ORDER BY c.customer_id;

```

48.

```

SELECT e.emp_name, e.department, e.salary,
       ROUND((SELECT AVG(x.salary) FROM employees x
              WHERE x.department = e.department), 2) AS dept_avg,
       ABS(e.salary - (SELECT AVG(x.salary) FROM employees x
                       WHERE x.department = e.department)) AS diff
FROM employees e
WHERE ABS(e.salary - (SELECT AVG(x.salary) FROM employees x
                      WHERE x.department = e.department)) = (
    SELECT MIN(ABS(y.salary - (
        SELECT AVG(z.salary) FROM employees z
        WHERE z.department = y.department
    )))

```

```

        FROM employees y
        WHERE y.department = e.department
    )
    ORDER BY e.department;

```

49.

```

SELECT c.customer_id, c.customer_name
FROM customers c
WHERE c.customer_id <> 201
AND NOT EXISTS (
    SELECT 1
    FROM orders o201
    WHERE o201.customer_id = 201
    AND NOT EXISTS (
        SELECT 1 FROM orders ox
        WHERE ox.customer_id = c.customer_id
        AND ox.product_id = o201.product_id
    )
)
ORDER BY c.customer_id;

```

50.

```

SELECT department, SUM(salary) AS dept_total
FROM employees
GROUP BY department
HAVING SUM(salary) > (
    SELECT MIN(dept_total)
    FROM (
        SELECT department, SUM(salary) AS dept_total
        FROM employees GROUP BY department
    ) x
)
ORDER BY dept_total DESC;

```

51.

```

SELECT e.emp_name,
       e.salary AS emp_salary,
       (SELECT m.emp_name FROM employees m
        WHERE m.emp_id = e.manager_id) AS manager_name,
       (SELECT m.salary FROM employees m
        WHERE m.emp_id = e.manager_id) AS manager_salary
FROM employees e
WHERE e.manager_id IS NOT NULL
AND e.salary > (
    SELECT m.salary FROM employees m
    WHERE m.emp_id = e.manager_id
)
ORDER BY emp_salary DESC;

```

52.

```

SELECT p.product_id, p.product_name,
       (SELECT COUNT(*) FROM orders o
        WHERE o.product_id = p.product_id) AS order_count
FROM products p
WHERE p.product_id IN (SELECT product_id FROM orders)
AND (SELECT COUNT(*) FROM orders o

```



```

        WHERE o.product_id = p.product_id) > (
        SELECT AVG(order_count)
        FROM (
            SELECT product_id, COUNT(*) AS order_count
            FROM orders GROUP BY product_id
        ) x
    );

```

53.

```

SELECT c.customer_id, c.customer_name,
       (SELECT MAX(o.amount) FROM orders o
        WHERE o.customer_id = c.customer_id) AS best_order
FROM customers c
WHERE (SELECT MAX(o.amount) FROM orders o
       WHERE o.customer_id = c.customer_id) = (
    SELECT MAX(amount) FROM orders
);

```

54.

```

SELECT emp_name, department, salary
FROM employees
WHERE department IN (
    SELECT dept_name
    FROM departments
    WHERE budget > (SELECT AVG(budget) FROM departments)
)
ORDER BY department, salary DESC;

```

55.

```

SELECT c.customer_id, c.customer_name,
       (SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id
        AND o.amount > (
            SELECT AVG(x.amount)
            FROM orders x
            WHERE x.customer_id = c.customer_id
        )) AS orders_above_avg
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
)
ORDER BY orders_above_avg DESC, c.customer_id;

```

56.

```

SELECT p.product_id, p.product_name,
       (SELECT MIN(c.city)
        FROM customers c
        WHERE c.customer_id IN (
            SELECT o.customer_id FROM orders o
            WHERE o.product_id = p.product_id
        )) AS only_city
FROM products p
WHERE p.product_id IN (SELECT product_id FROM orders)
AND (

```

```

SELECT COUNT(DISTINCT c.city)
FROM customers c
WHERE c.customer_id IN (
    SELECT o.customer_id FROM orders o
    WHERE o.product_id = p.product_id
)
) = 1;

```

57.

```

SELECT e.emp_name, e.department, e.salary
FROM employees e
WHERE e.salary = (
    SELECT MAX(x.salary)
    FROM employees x
    WHERE x.department = e.department
)
AND (
    SELECT COUNT(*)
    FROM employees x
    WHERE x.department = e.department
    AND x.salary = e.salary
) = 1
ORDER BY e.salary DESC;

```

58.

```

SELECT c.customer_id, c.customer_name,
       (SELECT SUM(o.amount) FROM orders o
        WHERE o.customer_id = c.customer_id) AS total_spend
FROM customers c
WHERE (SELECT SUM(o.amount) FROM orders o
       WHERE o.customer_id = c.customer_id) > 2 * (
    SELECT AVG(total_spend)
    FROM (
        SELECT customer_id, SUM(amount) AS total_spend
        FROM orders GROUP BY customer_id
    ) x
);

```

59.

```

SELECT e.department,
       (SELECT COUNT(*)
        FROM employees x
        WHERE x.department = e.department
        AND x.salary > (SELECT AVG(salary) FROM employees)
       ) AS above_avg_count
FROM employees e
GROUP BY e.department
ORDER BY above_avg_count DESC
LIMIT 1;

```

60.

```

SELECT e.emp_name, e.department, e.salary
FROM employees e
WHERE e.salary = (
    SELECT MAX(x.salary)
    FROM employees x

```

```
        WHERE x.department = e.department
    )
    AND (
        SELECT COUNT(DISTINCT x.salary)
        FROM employees x
        WHERE x.salary > e.salary
    ) >= 3
    ORDER BY e.salary DESC;
```